

KalaSetu: Product Requirements and System Architecture Document

Strategic Product Vision and Market Context

The Indian digital commerce ecosystem is currently undergoing a structural transformation of unprecedented scale, characterized by the systematic decentralization of monopolistic platform architectures and the rapid integration of rural micro, small, and medium enterprises (MSMEs). Macroeconomic indicators suggest that by 2025, the overarching e-commerce sector in India will surpass \$145 billion in valuation, with the direct-to-consumer (D2C) segment expanding at a formidable 40% compound annual growth rate to reach an estimated \$60 billion by 2030¹.

Concurrently, the proliferation of digital public infrastructure has laid a robust foundation for this growth; the Unified Payments Interface (UPI) processed 22.35 billion transactions in a single month in early 2026, while the Government e-Marketplace (GeM) achieved a cumulative Gross Merchandise Value (GMV) of Rs 18.4 lakh crore¹.

At the vanguard of this decentralization is the Open Network for Digital Commerce (ONDC), a population-scale digital public good initiated by the Department for Promotion of Industry and Internal Trade (DPIIT)². ONDC operates on an open-protocol network utilizing the Beckn specification, fundamentally decoupling the buyer-side application interface from the seller-side supply infrastructure³. This unbundling collapses the traditional duopoly model of e-commerce, driving merchant commission rates down from the extractive 25–35% typical of closed platforms to a highly competitive 3–5% on the open network³. The network's scale is accelerating rapidly; by early 2026, ONDC crossed 200 million cumulative transactions, supporting over 760,000 onboarded merchants and integrating 1.16 lakh retail sellers across 630 cities and towns³.

Despite these macroeconomic advancements, a profound digital divide persists, isolating India's estimated 7 million to 20 million rural artisans and craftspeople⁷. While physical internet penetration has surged—with over 95.15% of villages gaining 3G/4G connectivity under the Bharat Net initiative—the infrastructural reality remains fragile, and true digital literacy drastically lags behind raw network connectivity⁸. The traditional e-commerce onboarding process is inherently exclusionary; it mandates that a seller capture studio-quality photographs, digitally remove backgrounds, research and input complex search engine optimization (SEO) tags, navigate hierarchical product categories, and manually type extensive, text-heavy product specifications⁹. For a rural artisan producing hand-thrown terracotta pots, intricately woven textiles, or traditional wooden handicrafts, this form-heavy, English-first workflow acts as an insurmountable cognitive and operational barrier⁸.

KalaSetu is proposed as the definitive solution to this critical market failure. Positioned as an AI-powered product digitization layer and Seller Network Participant for ONDC, the system fundamentally replaces the traditional form-based onboarding paradigm with a multimodal "Zero-Form Commerce" approach⁹. The core product hypothesis posits that an artisan should only be required to intuitively show a physical product via a short smartphone video and speak a

natural description in their native regional language⁹. The underlying KalaSetu intelligence engine subsequently handles computational frame extraction, aesthetic evaluation, background removal, semantic attribute generation, and strict ONDC schema mapping, effectively reducing the time from product conception to a marketplace-ready, legally compliant catalog entry to under sixty seconds⁹.

Target Audience and Ethnographic HCI Constraints

To engineer a robust product from the ground up, the system architecture and User Interface (UI) must be inextricably rooted in the ethnographic realities of the target demographic. Rural entrepreneurs face a complex convergence of cognitive, socio-cultural, and infrastructural barriers that actively impede their participation in digital markets⁸.

Extensive ethnographic research in human-computer interaction (HCI) reveals that functionally illiterate or semi-literate individuals—a demographic encompassing approximately 750 million adults globally—interact with digital interfaces in ways that fundamentally diverge from the behaviors of highly literate users¹⁰. According to India's 75th National Sample Survey, only 4.4% of rural households owned a computer, and internet access was restricted to 14.9%, highlighting a stark lack of prior exposure to complex digital metaphors⁸. Cognitive limitations related to underdeveloped short-term memory and a reduced capacity for abstract, non-linear thinking dictate that low-literacy users profoundly struggle with hierarchical navigation trees, abstract iconography, and dense blocks of text¹⁰.

When confronted with a standard e-commerce cataloging web form, these users typically exhibit a narrowing of their visual field, reading methodically word-by-word, and frequently ignoring critical UI elements that fall outside of their immediate focal flux¹⁰. This phenomenon, sometimes termed "dysfunctional literacy" within HCI literature, results in slow, painful navigation and high abandonment rates when users are required to type out extensive product details¹⁰.

Consequently, the functional UI/UX requirements for KalaSetu must adhere to specific, research-backed design principles tailored for this exact demographic:

- **Linear, Unidirectional Workflows:** The system must eliminate nested menus and multi-tab forms entirely, replacing them with a guided, step-by-step sequential flow that limits cognitive overhead¹¹.
- **Representational Visuals over Abstraction:** The interface must utilize high-fidelity graphical feedback, hand-drawn graphics, or direct photographic representations rather than abstract standard icons (e.g., avoiding an abstract "disk" icon in favor of a literal representation of saving or finalizing)¹¹.
- **Voice-First Interaction:** The application must bypass text input dependencies by utilizing Voice User Interfaces (VUI) that support regional Indic languages, coupled with immediate audio feedback for error recovery and confirmation¹².

Functional Requirements: The Zero-Form Commerce Workflow

The product fundamentally reimagines the cataloging sequence, transmuting it from a

human-driven data entry task into a machine-driven inference and verification task. By synthesizing client-side heuristics with server-side generative AI, the system shifts the entire cognitive burden from the rural artisan to the cloud architecture.

Workflow Phase	Traditional E-Commerce Requirement	KalaSetu Zero-Form Implementation
Data Acquisition	Manual photography of multiple angles, manual typing of titles, descriptions, and metadata.	The artisan records a 5-10 second 360-degree video of the product and speaks a brief, natural description in their regional language.
Media Processing	Manual background removal via third-party editing tools; manual cropping and resizing.	Automated edge-based frame extraction, algorithmic blur filtering, and high-fidelity cloud AI background removal.
Catalog Enrichment	The seller independently researches SEO keywords, maps category trees, and estimates optimal pricing.	Multimodal AI automatically extracts color, material, and category, generates SEO-optimized tags, and calculates a suggested market price range.
Data Verification	The user submits a monolithic form and waits for opaque platform approval or rejection.	The AI identifies missing critical logistical data (e.g., volumetric weight) and asks 1-2 targeted voice questions. The artisan confirms via voice.
Network Output	A siloed listing published onto a single, closed-loop marketplace infrastructure.	An ONDC-compliant, cryptographically signed JSON-LD structured catalog, actively multicast to multiple buyer nodes across the open network.

This workflow ensures that the AI does not make unsupported, hallucinated guesses about

critical logistical information. If a five-second video cannot reliably provide the exact physical weight or packaged dimensions of a hand-carved wooden statue, the AI actively flags this uncertainty and initiates a localized voice prompt, adhering to a strict "zero-guessing" mandate for fulfillment parameters⁹.

Non-Functional Requirements: Edge Computing and Mobile Architecture

To mitigate network dependency and aggressive cloud computing costs, the initial phase of the product digitization pipeline must execute entirely at the edge, within the user's mobile browser.

WebAssembly and Mobile Memory Management

The frontend application is engineered as a Progressive Web App (PWA) utilizing Next.js, combined with WebAssembly (WASM) to execute C/C++ based multimedia processing libraries directly on the client device⁹. Specifically, a WASM port of FFmpeg (ffmpeg.wasm) is deployed to parse the raw video stream, extract individual candidate frames, and compress the media locally before any network transmission occurs, ensuring robust data security and minimizing bandwidth consumption¹⁷.

Deploying WASM on mobile devices, however, requires rigorous memory management strategies. While the WASM specification technically permits up to 4GB of memory allocation utilizing 32-bit pointers, mobile operating systems are notoriously hostile to high-memory web processes¹⁹. Forthcoming OS updates, such as Android 17, enforce strict per-app memory budgets that apply even to devices with 16GB of RAM, but are particularly punitive on budget smartphones equipped with only 4GB of physical memory²⁰.

If a background application or browser tab exceeds its predefined dynamic memory threshold, the Android operating system will attempt to aggressively compress the data, slowing the application, or invoke the Out-Of-Memory (OOM) killer to forcibly terminate the process silently²⁰. To prevent these silent crashes during video processing, the ffmpeg.wasm implementation within KalaSetu must process the video stream in highly constrained, discrete chunks, manually invoking garbage collection to clear frame buffers from the heap rather than attempting to load the entire video artifact into memory simultaneously²⁰.

Automated Frame Selection and Blur Detection Heuristics

Extracting every single frame of a 30-frames-per-second video over a 10-second duration yields 300 distinct images. Transmitting 300 uncompressed images to an expensive multimodal AI API is both financially prohibitive at scale and computationally redundant⁹. The edge architecture must algorithmically select the 1 to 3 highest-quality frames that optimally represent the product⁹.

Quality assessment at the edge begins with the immediate rejection of blurred frames caused by inherent hand tremors or poor camera autofocus²⁵. KalaSetu utilizes OpenCV.js to execute edge-based mathematical blur detection employing the variance of the Laplacian operator²⁷. The captured image is converted to a single-channel grayscale matrix, and the 3x3 Laplacian kernel is convolved across the pixels to measure the second derivative of the image, thereby highlighting regions of rapid intensity change associated with sharp physical boundaries²⁶.

The mathematical representation of this heuristic is defined as:

$$\sigma^2 = \text{Var}(\nabla^2 I)$$

A sharp, perfectly in-focus image contains numerous distinct edges, resulting in a wide spread of pixel intensity responses and a correspondingly high Laplacian variance²⁷. Conversely, a blurry image, lacking distinct structural edges, yields a tightly constrained spread of responses and a low variance²⁶.

Image Edge State	Laplacian Variance (σ^2)	System Routing Action
Highly Blurred (Unusable)	< 100 (Configurable Threshold)	Discard frame immediately at the edge to conserve memory.
Marginally Blurred	$100 - 300$	Retain in buffer temporarily, but heavily penalize the aesthetic score.
Sharp / In-Focus	> 300	Mark as an optimal candidate for cloud background removal and AI analysis.

While the Laplacian variance method is highly computationally efficient, it exhibits known sensitivities to high-frequency textures—such as woven cane baskets or complex embroidered textiles—which can artificially inflate the variance score even if the overall image is perceptually out of focus²⁵. Consequently, the selection algorithm must apply a secondary aesthetic and exposure pass. This involves analyzing the image histogram to detect severe under-exposure or over-exposure, ensuring the finalized candidate frames contain optimal lighting conditions and minimal physical occlusions²⁴.

Network Resilience and Background Synchronization

For rural artisans operating in regions characterized by intermittent or high-latency connectivity, the PWA must implement a fault-tolerant, offline-first architecture³⁰. If the user captures a product video while disconnected from the network, the application's Service Worker intercepts the outbound network request, serializing the processed optimal frames and encoded voice audio blobs directly into the browser's IndexedDB storage³². Utilizing the Background Sync API, the system queues a persistent synchronization event³⁰. When the mobile device subsequently regains a stable 3G/4G connection, the browser

automatically triggers the Service Worker to seamlessly flush the outbox queue³⁴. If a transmission attempt fails due to packet loss or gateway timeouts, the system executes an exponential backoff retry mechanism (e.g., retrying at escalating 2-second, 4-second, and 8-second intervals) to guarantee that the artisan's catalog data is eventually delivered to the cloud architecture without requiring the user to maintain an active, open application tab³³.

Multimodal AI and Computer Vision Processing

Once the optimized, selected frames successfully reach the cloud infrastructure, they undergo a sequential enhancement and data extraction process before being translated into the standardized commerce schema.

Intelligent Background Removal and Subject Isolation

The aesthetic quality of a product image heavily dictates consumer trust, perceived value, and ultimate conversion rates⁸. Raw images captured in rural workshops or domestic environments frequently contain visually distracting backgrounds that degrade the commercial viability of the listing. To rectify this, KalaSetu integrates the RMBG 2.0 model—developed by Bria AI—to flawlessly isolate the product subject³⁶.

RMBG 2.0 represents a significant architectural evolution in dichotomous image segmentation, operating on the advanced BiRefNet (Bilateral Reference Network) framework³⁸. This architecture incorporates a unique bilateral reference mechanism that intrinsically merges global semantic information (to understand the overarching context and localization of the scene) with highly precise gradient-level information (to detect sharp boundaries and intricate local details)³⁸.

This capability is uniquely critical for artisan products, which routinely feature complex fractal geometries, semi-transparent materials, or intricate fringing that confound standard algorithms. Unlike legacy binary mask generators that produce harsh, jagged, artificial cutouts, RMBG 2.0 generates sophisticated non-binary masks featuring 256 discrete levels of transparency⁴¹. This allows for exceptionally smooth, anti-aliased edges and the strict preservation of alpha channels (preserve_alpha: true), resulting in natural blending for diverse creative workflows⁴¹. Operating at a documented 90.14% overall success rate, and trained exclusively on legally licensed commercial data rather than scraped web imagery, the model ensures that the generated assets are entirely safe for enterprise deployment and fully compliant with stringent intellectual property laws³⁶. At an inference cost of approximately \$0.018 per generation, it provides a highly cost-effective utility while mitigating legal exposure⁴³. Furthermore, the model implementation utilizes visual_input_content_moderation and visual_output_content_moderation toggles to proactively flag sensitive inputs and ensure safe outputs for public network consumption⁴¹.

Multimodal Attribute Extraction and Hallucination Mitigation

The isolated, studio-quality product image, coupled with the transcribed text of the artisan's spoken description, is passed to a multimodal large language model (e.g., Gemini Pro Vision)⁹. The prompt engineering applied at this critical stage is strictly constrained and deterministic. The AI is explicitly instructed to act as a structured data extractor, analyzing the combined visual and textual inputs to autonomously generate:

1. An optimized, SEO-friendly Product Title that accurately reflects the item.
2. A rich, detailed product description highlighting craftsmanship.
3. Categorization tags strictly mapped to the approved ONDC taxonomy.
4. Visible material characteristics (e.g., identifying "terracotta," "handloom cotton," or "brass").
5. A suggested pricing range, synthesized from comparable market data, raw material costs, and expected craftsmanship margins⁹.

A known, critical flaw in applying generative AI to e-commerce is hallucination—the model's tendency to confidently guess attributes it cannot empirically verify⁹. For instance, a multimodal vision model cannot accurately ascertain the exact physical weight or packaged dimensions of a wooden handicraft purely from a photograph, yet these specific metrics are absolutely mandatory for calculating accurate logistics costs, volumetric weight, and shipping parameters across the ONDC network⁹.

To effectively counter this, the KalaSetu system enforces a strict "Zero-Guessing" policy for all critical logistical and statutory fields⁹. The AI is programmed to generate an explicit Confidence Score for every attribute it attempts to extract⁹. If the confidence score for a specific attribute (such as weight or dimensions) falls below a 95% threshold, or if the field is deemed structurally impossible to determine via visual analysis alone, the system proactively flags the attribute as missing_information rather than populating the catalog with fabricated data⁹.

The Voice Interaction and NLP Layer

When the AI architecture detects missing_information regarding critical catalog attributes, the UI deliberately does not present the artisan with a standard text-input form. Instead, the application seamlessly transitions to a conversational voice interface, soliciting the specific missing data points directly from the user through localized audio prompts⁹.

Real-Time Streaming ASR Integration

To adequately support the highly diverse linguistic landscape of the Indian subcontinent, KalaSetu integrates directly with Bhashini, the Government of India's national language technology platform spearheaded by the Ministry of Electronics and Information Technology (MeitY)⁴⁵. Bhashini provides specialized speech-to-text (ASR), translation, and text-to-speech (TTS) models trained explicitly on Indic languages, ensuring high accuracy for regional dialects and accents⁴⁶. Access to these enterprise-grade APIs requires a subscription, with commercial SLAs offering speech synthesis of up to 50,000 characters per day at a baseline cost of ₹250 per month for users within India⁴⁸.

To achieve the human-like conversational latency required to prevent user frustration, the system bypasses standard REST APIs and utilizes Bhashini's WebSocket API for real-time, full-duplex streaming⁴⁹. The audio captured from the mobile device's microphone is transmitted continuously as binary frames, encoded in linear16 format at a sampling rate of 16000 Hz, with a recommended streaming chunk size of 100–200 milliseconds to minimize latency⁵⁰.

The WebSocket connection is securely initialized with a JSON control payload that explicitly configures the server-side Voice Activity Detection (VAD) engine parameters:

```

JSON
{
  "event" "start"
  "language" "hi"
  "useVad" true
  "inputEncoding"
  "encoding" "linear16"
  "samplingRate" 16000
  "bitsPerSample" 16
  "numChannels" 1
}
{
  "vadConfig"
  "pStart" 0.6
  "pauseMs" 400
  "endMs" 1200
  "prerollMs" 240
  "frameDurationMs" 32
}
{
  "interimIntervalMs" 200
}

```

Voice Activity Detection (VAD) State Machine

A prevalent usability failure mode with voice interfaces deployed among low-literacy populations is that users frequently pause mid-sentence to think, gather their thoughts, or physically inspect their product¹². Legacy voice systems interpret these minor pauses as the end of the input, prematurely cutting the user off and causing immense frustration¹⁴. Bhashini's server-side VAD engine circumvents this by utilizing a highly tuned Finite State Machine (FSM) to analyze acoustic probabilities and intelligently manage speech segmentation⁵⁰.

The FSM navigates five distinct operational states based on continuous audio analysis:

1. **IDLE:** The default state when no speech is detected. Background noise is successfully ignored⁵⁰.
2. **STARTING:** Triggered when the probability of speech exceeds the configured pStart threshold (0.6). Audio is temporarily buffered (prerollMs: 240) to prevent clipping the initial syllable of the user's utterance⁵⁰.
3. **SPEAKING:** Confirmed active speech is streamed continuously to the ASR decoding engine⁵⁰.
4. **PAUSED:** If acoustic silence is detected for the duration of pauseMs (400ms), the system enters a paused state. Crucially, it continues collecting audio, correctly anticipating that

the user is merely taking a breath or organizing their thoughts⁵⁰.

5. **ENDING:** Only if the continuous silence exceeds the endMs threshold (1200ms) is the utterance officially considered complete, at which point the final, consolidated transcript is generated and the AI evaluates the textual response⁵⁰.

This sophisticated, forgiving VAD logic allows the rural artisan to speak naturally, pause to handle their product, and resume speaking without the digital interface abruptly severing the conversational flow. Once the final transcript is returned, the system's NLP layer parses the numerical values (e.g., "It weighs exactly two kilograms") and dynamically maps them to the pending catalog schema.

Database Architecture and Query Optimization

The fully generated and verified catalog data must be persisted in a highly available, scalable database architecture capable of supporting rapid read-heavy operations from the network. KalaSetu utilizes PostgreSQL, heavily leveraging its advanced JSONB data type to store the complex, highly nested, and schema-less payloads that are characteristic of the Beckn protocol specifications⁵¹.

JSONB Indexing Strategies and Performance

The Beckn protocol dictates a deeply nested, hierarchical structure for retail catalogs, containing arbitrary arrays of items, detailed descriptors, complex price objects, and variable tags⁵³. A pervasive architectural anti-pattern when developing with JSONB columns is applying a blanket Generalized Inverted Index (GIN) across the entire column and assuming it will automatically accelerate all conceivable query types⁵¹.

A default GIN index (utilizing jsonb_ops) is highly optimized to support containment operators (@>, ?, ?|, ?&), but it fundamentally does not support basic equality checks using the standard ->> operator⁵².

If the application executes a standard query to find all active catalogs for a specific seller:

```
SQL
SELECT * FROM          WHERE          ->>'status' = 'active'
```

The PostgreSQL query planner will completely ignore the default GIN index and execute a highly costly sequential scan across the entire table⁵². On a table containing millions of complex JSON documents, reading every row destroys database performance and severely spikes I/O costs⁵¹. Furthermore, maintaining a massive GIN index is computationally expensive; every INSERT or UPDATE transaction requires the database engine to decompose the entire JSON document to update the index entries, which can degrade overall write throughput by 30-50% on write-heavy tables⁵¹.

To achieve optimal performance and scalability for KalaSetu's backend, the database architecture relies on a targeted, hybrid indexing strategy:

- **Targeted Expression Indexes:** For highly specific, frequently queried exact-match keys (such as checking a specific seller ID or verifying a boolean status flag), a standard B-tree expression index is deployed, offering vastly superior speed for simple equality checks⁵¹.
- **Partial Indexes:** To strictly limit the total index size and significantly accelerate read times, partial indexes are created to only index the subset of rows that are actively queried. For example, indexing only active product listings while ignoring archived ones:

SQL

```
CREATE INDEX CONCURRENTLY idx_catalogs_active
ON catalogs (metadata) -->'seller_id'
WHERE (metadata -->'status' = 'active')
```

This architectural decision ensures that massive database writes to archived, deleted, or pending rows bypass the index entirely, keeping memory overhead exceptionally low and preserving write throughput⁵².

- **Path Ops for Containment:** For complex discovery functionalities where ONDC buyer nodes query multiple nested attributes simultaneously (e.g., filtering by a specific material tag within an array), a GIN index utilizing the specialized jsonb_path_ops operator class is implemented⁵¹. jsonb_path_ops hashes the full hierarchical path to each value rather than indexing every individual key-value pair separately, resulting in an index that is mathematically 3-4x smaller and significantly faster than the default jsonb_ops implementation⁵².

ONDC Protocol Alignment and Technical Compliance

KalaSetu's ultimate software output is not merely a structured database entry, but an actively multicast catalog operating securely on the Open Network for Digital Commerce. The system architecture must seamlessly interface with the ONDC Protocol (the Network Extension Layer) and the underlying Beckn Protocol (the Base Layer)⁵⁷.

Network Participant Topologies and Agreements

Within the ONDC ecosystem, all network participants are strictly categorized by their operational roles, defined under Chapter 2 of the ONDC Network Policy⁵⁸. KalaSetu functions primarily as a technological enabler for the Seller Side Application. Depending on the desired go-to-market strategy, KalaSetu can operate under distinct participant topologies:

1. **Inventory Seller Node (ISN):** The system acts as the direct, dedicated application for an individual artisan who produces and sells their own unique inventory⁶⁰. Under this topology, KalaSetu or the artisan themselves acts as the definitive "seller on record" in the digital registry⁶⁰.
2. **Marketplace Seller Node (MSN):** The system aggregates multiple distinct artisans under a single, unified node⁶⁰. Here, the MSN provides the technological bridge and digitization layer, but the individual third-party artisans remain the legal sellers on record for their respective goods⁶⁰.
3. **Technology Service Provider (TSP):** KalaSetu operates in a purely B2B SaaS capacity, providing the video-to-catalog AI infrastructure as a modular plug-in for existing,

established ONDC seller apps (such as GoFrugal, SellerApp, or Mystore)⁵⁸. Regardless of the topology, every participant must execute the overarching Network Participant Agreement (NPA), binding them to the ONDC Network Policy, which mandates fair competition, data confidentiality, and rapid grievance resolution through Online Dispute Resolution (ODR) mechanisms⁵⁹.

Beckn Schema Mapping and Cryptographic Security

The core mapping engine of the application translates the verified AI outputs into the precise Beckn JSON-LD specifications required by the network⁵³. The ONDC Retail (RET) Specifications accommodate highly comprehensive catalog parameters⁶⁴. Under the advanced Beckn v2.1 schema, per-commitment line-item details are meticulously managed through CommitmentAttributes⁶⁵. This crucial update consolidates previously disparate fields—such as exact quantities, universal unit codes, base pricing, estimated value ranges, and special handling instructions (e.g., specific buyer notes like "fragile terracotta, handle with care")—into a uniform, queryable structure⁶⁵.

The system maps the AI-extracted data to specific Beckn structural nodes:

- **descriptor.name:** The AI-generated, SEO-optimized product title⁶⁶.
- **descriptor.code:** The Harmonized System of Nomenclature (HSN) code accurately classifying the good (e.g., utilizing 44219090 for specific wooden handicrafts)⁶⁷.
- **price.currency & price.value:** The finalized, artisan-approved retail price⁵⁴.
- **tags:** Extended metadata arrays utilized for highly specific searchable filters⁶⁷.

To guarantee the integrity and origin of these catalog broadcasts, all HTTP communications over the Beckn APIs require rigorous authentication⁷¹. KalaSetu implements the mandated Ed25519 Signature Scheme⁷¹. For every outgoing payload, the system generates a BLAKE-512 cryptographic digest of the request body, appends UNIX timestamps for the created and expires fields to prevent replay attacks, and signs the concatenated string using the node's registered private key⁷¹. This Base64-encoded signature is then injected into the standard Authorization header, ensuring full protocol compliance⁷¹.

Legal Metrology Automation

A paramount regulatory hurdle for all forms of digital commerce in India is absolute compliance with the Legal Metrology (Packaged Commodities) Amendment Rules 2026⁷². This federal law explicitly mandates that e-commerce entities must display precise, unambiguous product information *before* a transaction is completed, protecting consumers from misleading or unfair trade practices⁷².

Crucially, from an engineering perspective, this is a complex data structuring problem, not a simple front-end UI labeling task⁷². The required attributes cannot merely be buried inside a free-text descriptive paragraph; they must exist as clean, searchable, and sortable database fields that propagate uniformly across the entire decentralized network⁷².

KalaSetu's internal schema validation engine enforces the strict inclusion of these mandatory Legal Metrology attributes on every single catalog item before network broadcast:

- **Net Quantity:** The exact weight, volume, or numerical count of the product⁷².

- **Maximum Retail Price (MRP):** The definitive maximum price, strictly inclusive of all applicable taxes⁷².
- **Manufacturer / Packer Details:** The registered name and physical address of the artisan or cooperative⁷².
- **Country of Origin:** Mandatory for any imported materials, implemented as a discrete, filterable tag rather than plain text⁷².
- **Date of Manufacturing / Expiry:** Sourced directly from the artisan during the voice-verification phase⁷³.

Failure to provide these fields accurately exposes both the artisan and the host node to severe legal penalties, including fines scaling from ₹25,000 to over ₹50,000 for repeated offenses, product seizure, and permanent suspension from the ONDC network⁷³. By embedding these strict requirements directly into the generative AI prompt constraints and the subsequent VUI verification checklist, KalaSetu mathematically ensures that no product is pushed to the ONDC gateway without meeting all statutory requirements⁷².

Error Handling and Policy as Code

Interactions across the ONDC network are fundamentally asynchronous, involving a complex choreography of primary requests and subsequent callbacks (e.g., a search request is eventually followed by an `on_search` callback)⁵. As the artisan's catalog is broadcast to various buyer applications via the gateway, numerous network validations occur simultaneously⁵. KalaSetu implements highly robust, automated error handling tailored specifically to the standardized ONDC error code registry, providing actionable, plain-language feedback to the artisan if an issue arises⁷⁶.

- **Code 20000 (Invalid Catalog):** Indicates the payload failed JSON schema validation at the destination buyer node⁷⁶. KalaSetu aggressively prevents this by passing all outbound catalogs through an internal, highly optimized SchemaValidator plugin prior to transmission⁷⁷.
- **Code 30000 (Invalid Request):** The outbound request failed to meet the strict API contract specifications⁷⁶.
- **Code 30004 (Item Not Found):** Triggered when a buyer attempts to interact with or purchase a product that the artisan recently deleted or marked as out of stock⁷⁶.
- **Code 10001 (Invalid Signature):** The gateway rejected the request due to cryptographic failure, prompting an immediate token refresh and re-signing sequence⁷⁶.

Furthermore, the system embraces a modern "Policy as Code" framework to automate absolute compliance with the strict ONDC Network Policy⁷⁹. The Network Participant Agreement dictates that all entities must act fairly, explicitly avoid selling counterfeit or prohibited goods, and ensure consumer safety⁵⁹. During the initial background removal phase utilizing RMBG 2.0, the system triggers the `visual_input_content_moderation` toggle to automatically flag sensitive, illegal, or prohibited inputs before processing even begins, ensuring that illicit items are rejected securely at the edge before unnecessarily consuming expensive cloud resources or violating network policies⁴¹.

Success Metrics and Key Performance Indicators (KPIs)

To evaluate the efficacy and market penetration of the KalaSetu platform, continuous monitoring of specific, actionable KPIs is required, aligning with the broader growth metrics of the ONDC network.

Metric Category	Key Performance Indicator (KPI)	Target Benchmark
User Acquisition & Engagement	Time-to-Catalog (from video start to ONDC JSON generation)	< seconds
	VAD Drop-off Rate (users abandoning during voice prompts)	<
	Monthly Active Artisans Onboarded	Month-over-Month Growth >
System & Technical Performance	Edge Frame Extraction Success Rate (evading OOM kills)	>
	SchemaValidator Compliance Rate (avoiding Code 20000)	100%
	Average AI Hallucination Flag Rate (triggering Voice UI)	< of total fields
Commercial Output	Number of ONDC Orders Facilitated	Tracking against ONDC's 7.6+ Million monthly volume ⁸⁰
	Legal Metrology Compliance Audit Pass	100%

	Rate	
--	------	--

Conclusion

KalaSetu represents a fundamental, highly engineered paradigm shift in e-commerce onboarding for historically marginalized and digitally disconnected communities. By synthesizing advanced client-side edge computing (WASM memory management, OpenCV Laplacian blur detection), state-of-the-art computer vision models (RMBG 2.0 BiRefNet architecture), real-time streaming acoustic NLP (Bhashini WebSocket ASR), and robust asynchronous data architectures (PostgreSQL JSONB targeted indexing), the system systematically dismantles the cognitive and infrastructural barriers inherent in traditional text-heavy forms.

The strategic, technical alignment with the asynchronous ONDC Beckn protocol, combined with an uncompromising, automated adherence to India's strict Legal Metrology regulations, ensures that the resulting output is not just a digital artifact, but a fully compliant, highly interoperable commercial asset. By transforming the arduous digitization workflow into a frictionless, accessible "Show, Speak, and Sell" experience, KalaSetu successfully bridges the vast divide between rural Indian craftsmanship and the explosive growth of the national digital economy.

Works cited

1. India's E-commerce Boom: Growth, Trends & Future Prospects | IBEF, <https://www.ibef.org/industry/ecommerce>
2. Open Network for Digital Commerce (ONDC), <https://ondc.org/>
3. DEMOCRATIZING DIGITAL COMMERCE: ONDC AND THE, <https://www.impriindia.com/insights/policy-update/democratizing-digital-commerce-ondc-and-the-future-of-market-competition-in-india-2026/>
4. Open Network for Digital Commerce (ONDC), <https://www.ondc.org/>
5. README.md - becn/protocol-specifications - GitHub, <https://github.com/becn/protocol-specifications/blob/master/README.md>
6. ONDC Enables Fair, Transparent and Inclusive E-Commerce ... - PIB, <https://www.pib.gov.in/PressReleasePage.aspx?PRID=2204664>
7. Indian artisans are still missing from e-commerce platforms, <https://www.sattva.co.in/news-media/indian-artisans-are-still-missing-from-e-commerce-platforms/>
8. ADOPTION OF E-COMMERCE PLATFORMS FOR RURAL, <https://www.jetir.org/papers/JETIRGU06131.pdf>
9. KalaSetu_Idea_Analysis.pdf
10. Interface design guidelines for low literature users - IPCA, <https://ciencipca.ipca.pt/bitstreams/0ff53342-b834-4a99-a114-3082cd636e02/download>
11. (PDF) Designing User Interfaces for Illiterate and Semi-Literate Users, https://www.researchgate.net/publication/370837718_Designing_User_Interfaces_for_Illiterate_and_Semi-Literate_Users_A_Systematic_Review_and_Future_Research_Agenda

12. A Review on Voice User Interface for Illiterate People - Academia.edu,
https://www.academia.edu/9685597/A_Review_on_Voice_User_Interface_for_Illiterate_People
13. Interface design guidelines for low literature users: a literature review,
https://www.researchgate.net/publication/370383746_Interface_design_guidelines_for_low_literature_users_a_literature_review
14. Experiences designing a voice interface for rural India - ResearchGate,
https://www.researchgate.net/publication/224382217_Experiences_designing_a_voice_interface_for_rural_India
15. User Interface Design for Low-literate and Novice Users,
<https://www.emerald.com/fthci/article/8/1/1/1320900/User-Interface-Design-for-Low-literate-and-Novice>
16. Designing interactive voice response (IVR) interfaces - IBM Research,
<https://research.ibm.com/publications/designing-interactive-voice-response-ivr-interfaces-localisation-for-low-literacy-users>
17. ffmpeg.wasm | ffmpeg.wasm, <https://ffmpegwasm.netlify.app/>
18. tejaswigowda/ffmpeg-webCLI: A browser-based video ... - GitHub,
<https://github.com/tejaswigowda/ffmpeg-webCLI>
19. Up to 4GB of memory in WebAssembly - V8 JavaScript engine,
<https://v8.dev/blog/4gb-wasm-memory>
20. Android 17 puts app RAM on a budget and can close memory,
<https://news.lavx.hu/article/android-17-puts-app-ram-on-a-budget-and-can-close-memory-hungry-apps>
21. Google clamps down on Android app RAM usage amid AI memory,
<https://www.tomshardware.com/phones/android/google-clamps-down-on-android-app-ram-usage-amid-ai-memory-crisis-developers-have-until-february-2027-to-adapt-to-new-memory-optimizing-rules>
22. but I think each browser tab has a memory limit of 4gb? this means a,
<https://news.ycombinator.com/item?id=40844441>
23. Chrome is unable to grow WebAssembly Memory sizes on Android,
<https://issues.chromium.org/40747423>
24. A Novel Frame-Selection Metric for Video Inpainting to Enhance,
https://www.researchgate.net/publication/380564248_A_Novel_Frame-Selection_Metric_for_Video_Inpainting_to_Enhance_Urban_Feature_Extraction
25. Detect image blur using Laplacian variance (OpenCV) · GitHub,
<https://github.com/MariaDominic3014/blur-detection-project>
26. How to Check for Blurry Images in Your Dataset Using the Laplacian,
<https://www.geeksforgeeks.org/computer-vision/how-to-check-for-blurry-images-in-your-dataset-using-the-laplacian-method/>
27. Blur detection with OpenCV - PyImageSearch,
<https://pyimagesearch.com/2015/09/07/blur-detection-with-opencv/>
28. Blur image detection using Laplacian operator and Open-CV,
https://www.researchgate.net/publication/315919131_Blur_image_detection_using_Laplacian_operator_and_Open-CV
29. A Novel Frame-Selection Metric for Video Inpainting to Enhance,

- <https://pmc.ncbi.nlm.nih.gov/articles/PMC11125042/>
30. Data Synchronization in PWAs: Offline-First Strategies and Conflict, <https://gtcsys.com/insights/blogs/data-synchronization-in-pwas-offline-first-strategies-and-conflict-resolution/>
 31. Offline Support and Progressive Web Apps (PWAs) - DEV Community, <https://dev.to/zeeshanali0704/frontend-system-design-offline-support-and-progressive-web-apps-pwas-4k8m>
 32. workbox-background-sync | Modules - Chrome for Developers, <https://developer.chrome.com/docs/workbox/modules/workbox-background-sync>
 33. Advanced PWA Guide: Offline Caching, Push Notifications, <https://rishikc.com/articles/advanced-pwa-features-offline-push-background-sync/>
 34. Offline and background operation - Progressive web apps | MDN, https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Offline_and_background_operation
 35. PWA Advanced Implementation Guide - Service Worker Cache, https://hidekazu-konishi.com/entry/pwa_advanced_implementation_guide.html
 36. RMBG 2.0: Breaking New Ground in Background Removal, https://dev.to/nick_talbert_fe9a5eebf941/rmbg-20-breaking-new-ground-in-background-removal-hoi
 37. RMBG-2.0 for background removal
 38. Introducing the RMBG v2.0 Model – The Next Generation in Background Removal from Images, <https://bria.ai/introducing-the-rmbg-v2-0-model-the-next-generation-in-background-removal-from-images>
 39. RMBG-2.0: Open Source Background Removal Model | Notebooks, <https://hyper.ai/tutorials/36475>
 40. Exploring Image Background Removal Using RMBG v2.0, <https://www.analyticsvidhya.com/blog/2025/03/rmbg-v2-0/>
 41. Bria RMBG 2.0 API - Background Removal Model - Segmind, <https://www.segmind.com/models/bria-remove-background>
 42. briaai/RMBG-2.0 - Hugging Face, <https://huggingface.co/briaai/RMBG-2.0>
 43. Bria RMBG 2.0: Professional Image Background Removal Tool - Fal.ai, <https://fal.ai/models/fal-ai/bria/background/remove>
 44. ONDC Seller: Detailed Analysis and Registering Your Business, <https://pointnxt.com/blog/ondc-sellers-guide/>
 45. Bhashini-AI4Bharat Textual Language Detection v1.0 - AIKosh, https://aikosh.indiaai.gov.in/home/model-tryout/details/bhashini_ai4bharat_textual_language_detection_v1_0.html
 46. Bhashini API: real-time STT, translation, TTS - CodePrism, <https://codeprismtechnologies.com/case-studies/bhashini-multilingual-stt-tts-pipeline>
 47. Bhashini, <https://bhashini.gov.in/en/>
 48. Server Subscription - Bhashini AI Solutions, <https://www.bhashini.ai/speak/subscription>
 49. WebSocket ASR API - Bhashini APIs - GitBook, <https://dibd-bhashini.gitbook.io/bhashini-apis/websocket-asr-api>

50. Bhashini.ai Streaming Speech-to-Text (STT) WebSocket,
<https://www.bhashini.ai/docs/stt>
51. PostgreSQL JSONB Indexing: GIN, Expression & Partial Index,
<https://mydba.dev/blog/postgres-jsonb-indexing>
52. PostgreSQL JSONB Indexing: GIN, Expression & Partial Index,
https://dev.to/philip_mccclarence_2ef9475/postgresql-jsonb-indexing-gin-expression-partial-index-strategies-i11
53. search query specification · Wiki · Applications / Sargasso / Agri-Beckn,
https://git.startinblox.com/applications/sargasso/agri-beckn/-/wikis/our-beckn-proposal/search-query-specification/diff?version_id=f3b87fca7dd86e18b4dd687579c7d23dbdd0ce44&view=parallel
54. ONDC-Protocol-Specs/protocol-specifications/core/v0/api/core.yaml,
<https://github.com/ONDC-Official/ONDC-Protocol-Specs/blob/master/protocol-specifications/core/v0/api/core.yaml>
55. Documentation: 18: 65.4. GIN Indexes - PostgreSQL,
<https://www.postgresql.org/docs/current/gin.html>
56. PostgreSQL as a JSON database: Advanced patterns and best,
<https://aws.amazon.com/blogs/database/postgresql-as-a-json-database-advanced-patterns-and-best-practices/>
57. GitHub - ONDC-Official/protocol-network-extension,
<https://github.com/ONDC-Official/protocol-network-extension>
58. ONDC | Micro, Small & Medium Enterprises, Government of West,
<https://msme.wb.gov.in/ondc>
59. ONDC Network Policy - Resources & Knowledge Base,
<https://resources.ondc.org/ondc-network-policy>
60. Open Network for Digital Commerce (ONDC) - Argus Partners,
<https://www.argus-p.com/papers-publications/thought-paper/open-network-for-digital-commerce-ondc-issues-and-challenges-for-payment-aggregators/>
61. Seller Network Participants - ONDC,
<https://www.ondc.org/pages/seller-network-participants.html>
62. ONDC Network Policy · Resources,
<https://www.ondc.org/pages/resources-network-policy.html>
63. ONDC Network Participant Agreement | PDF | Taxes - Scribd,
<https://www.scribd.com/document/900625007/DRAFT-ONDC-Draft-Network-Participant-Agreement-Trilegal>
64. ONDC-Official/ONDC-RET-Specifications - GitHub,
<https://github.com/ONDC-Official/ONDC-RET-Specifications>
65. RetailCommitment — schema.nfh.global,
<https://schema.nfh.global/RetailCommitment>
66. Integrate ONDC with just a few lines of code - Medium,
<https://life-of-utkarsh.medium.com/integrate-ondc-with-just-a-few-lines-of-code-6bf25b622294>
67. UEI Implementation Guide - EV Charging - GitHub,
https://github.com/beckn/missions/blob/main/UEI/implementation-guides/EV-charging/implementation_guide_charging.md

68. HSN Code for Wooden Handicraft - GST Rate, Export Rules & ITC,
<https://impexkit.com/hsn-code/wooden-handicraft/>
69. Supported structured data attributes and values - Google Help,
<https://support.google.com/merchants/answer/6386198?hl=en>
70. Tags : The edge of beckn | CWG Working Draft #316 - GitHub,
<https://github.com/beckn/protocol-specifications/discussions/316>
71. developer-docs/registry/signing-verification.md at main - GitHub,
<https://github.com/ONDC-Official/developer-docs/blob/main/registry/signing-verification.md>
72. Legal Metrology E-Commerce Rules 2026: What D2C Must Fix,
<https://www.braincuber.com/blog/legal-metrology-ecommerce-rules-d2c>
73. Legal Metrology Rules for E-commerce Sellers in India,
<https://www.diligencecertification.com/legal-metrology-for-e-commerce/>
74. Legal Metrology Compliance for E-commerce in India,
<https://legalmetrologyindia.com/blog/legal-metrology-compliance-for-e-commerce-in-india/>
75. Beckn Adaptor API Setup Guide | PDF - Scribd,
<https://www.scribd.com/document/736827442/Beckn-L1-API-Technical-Configuration-Doc>
76. error-codes.md - ONDC-Official/developer-docs - GitHub,
<https://github.com/ONDC-Official/developer-docs/blob/main/protocol-network-extension/error-codes.md>
77. schemavalidator package - github.com/ONDC-Official/automation,
<https://pkg.go.dev/github.com/ONDC-Official/automation-beckn-onix/pkg/plugin/implementation/schemavalidator>
78. ONDC-Official/automation-beckn-onix - GitHub,
<https://github.com/ONDC-Official/automation-beckn-onix>
79. Compliance for ONDC Seller Apps | PDF | E Commerce - Scribd,
<https://www.scribd.com/document/900623458/DRAFT-Network-Policies-Seller-Requirements>
80. Publications · Resources · ONDC Network,
<https://www.ondc.org/pages/resources-publications.html>